**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ**
**НАЦІОНАЛЬНОМУ УНІВЕРСИТЕТІ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”**

**Кафедра систем штучного інтелекту**

# Лабораторна робота №5

з дисципліни
«Операційні системи»

**Виконав:**
студент групи КН-114
Добрій Назарій

**Викладач:**
Кривенчук Ю.П.

Львів – 2020 р.

**Тема**. Робота з динамічними бібліотеками в ОС Windows

**Мета**. Ознайомитися з динамічно-зв’язувальними бібліотеками (Dynamic-link library) в ОС Windows. Навчитися реалізовувати динамічно-зв’язувальні бібліотеки.

Завдання.

1. Реалізувати **лабораторну роботу №4** у вигляді динамічно-зв’язувальної бібліотеки.
2. Запустити створену бібліотеку з командної стрічки (cmd.exe) за допомогою rundll32.exe.
3. Створити окрему програму і реалізувати статичний зв'язок між програмою та бібліотекою.
4. Створити окрему програму і реалізувати динамічний зв'язок між програмою та бібліотекою.
5. Експортувати головну функцію бібліотеки під іншим іменем.
6. Результати виконання роботи відобразити у звіті.

Створення dll бібліотеки:

Створення Header файлу "Synchronization.h"

```
#include <windows.h>
#include <algorithm>
#include <vector>
#include <ctime>

using namespace std;

//dll macros
#ifdef SYNCHONIZATION_EXPORTS
#define SYNCHONIZATION_API _declspec(dllimport)
#else
#define SYNCHONIZATION_API _declspec(dllexport)
#endif // SYNCHONIZATION_EXPORTS

extern "C" SYNCHONIZATION_API int selectionSort_start();

#endif // SYNCHONIZATION_H
```

Підключення Header файлу:

```
#include "Synchronization.h"

BOOL APIENTRY DllMain(HMODULE hmodule,
    DWORD ul_reason_for_call,
    LPVOID lpReserved) {
    switch (ul_reason_for_call) {
    case DLL_PROCESS_ATTACH:
    case DLL_PROCESS_DETACH:
    case DLL_THREAD_ATTACH:
    case DLL_THREAD_DETACH:
        break;
    }

    return TRUE;
}
```

Створення файлу "Synchronization.cpp" та функцію яку будем викликати з інших програм - selectionSort_start() :

```cpp
#define _CRT_SECURE_NO_WARNINGS
#include "Synchronization.h"
#include <iostream>
#include <windows.h>
#include <algorithm>
#include <vector>
#include <ctime>

using namespace std;

HANDLE hThreads[8];
HANDLE hThreadOpen[8];
HANDLE hMutex;

vector <int> ThreadFuncTakeNums;

volatile LONG TotalThreadCount = 0;

DWORD WINAPI selectionSort(void* param) {
    vector<int>& temp = *(vector<int>*) param;
    int min;

    WaitForSingleObject(hMutex, INFINITE);
    for (int i = 0; i < temp.size(); i++) {
        ThreadFuncTakeNums.push_back(temp[i]);
    }

    for (int i = 0; i < temp.size() - 1; i++) {

        min = i;
        for (int j = i + 1; j < temp.size(); j++)
        for (int j = i + 1; j < temp.size(); j++)
            if (temp[j] < temp[min])
                min = j;
        swap(temp[i], temp[min]);

    }
    ReleaseMutex(hMutex);

    return 0;
}

vector<int> complete(vector<vector<int>> arr, int n, int k) {
    vector<int> completed(n);
    for (int i = 0; i < n; i++) {
        int min = 1000000;
        int th = -1;
        for (int j = 0; j < k; j++) {
            if (!arr[j].empty() && arr[j][0] < min) {
                min = arr[j][0];
                th = j;
            }
        }
        completed[n - i - 1] = min;
        arr[th].erase(arr[th].begin());
    }
    return completed;
}

int selectionSort_start() {
    AllocConsole();
    freopen("CONIN$", "r", stdin);
    freopen("CONIN$", "r", stdin);
    freopen("CONOUT$", "w", stdout);
    freopen("CONIN$", "r", stderr);

    srand(time(NULL));

    DWORD dwIncre;

    //creating mutex
    hMutex = CreateMutex(
        NULL,
        FALSE,
        NULL
    );

    int elem_num, thread_num;
    vector<vector<int>> arr;

    cout << "Enter the number of elements: ";
    cin >> elem_num;
    vector<int> input(elem_num);
    for (int i = 0; i < elem_num; ++i) {
        input[i] = rand() % elem_num + 1;
        cout << input[i] << " ";
    }
    cout << endl;

    cout << "Enter the number of threads: ";
    cin >> thread_num;
    for (int i = 0; i < thread_num; ++i) {
        vector<int> tmp;
```

```
        vector<int> tmp;
        for (int j = i * elem_num / thread_num; j < (i + 1) * elem_num / thread_num; j++) {
            tmp.push_back(input[j]);
        }
        arr.push_back(tmp);
    }

    int last_interation = 0;

    for (int i = 0; i < thread_num; i++) {
        int ThreadPriority;

        DWORD ThreadID;

        HANDLE thread = CreateThread(NULL, 0, selectionSort, &arr[i], 0, &ThreadID);
        hThreads[i] = thread;

        cout << "Number of thread - " << i + 1 << endl;

        //ThreadID
        cout << "Thread ID = " << ThreadID << endl;

        cout << "Values, which take this " << ThreadID << " thread:" << endl;

        for (int k = last_interation; k < ThreadFuncTakeNums.size(); k++)
        {
            cout << ThreadFuncTakeNums[k] << " ";
        }
        cout << endl;

        if (hThreads[i] == NULL)
```

```
        if (hThreads[i] == NULL)
        {
            cout << "Thread Error" << endl;
            ExitProcess(1);
        }

        //Opend Thread
        HANDLE hOpenThread = OpenThread(
            THREAD_SET_LIMITED_INFORMATION,
            FALSE,
            ThreadID
        );

        //interlock function
        dwIncre = InterlockedIncrement((LPLONG)&TotalThreadCount);

        hThreadOpen[i] = hOpenThread;

        if (NULL == hOpenThread)
        {
            cout << "Open Thread Function Failed & Error - " << GetLastError() << endl;
            ExitProcess(2);
        }

        // Set Thread Priority
        int mode;
        cout << "Set priority: " << endl;
        cout << "Enter index of priority Idle(1), Below Normal(2), Normal(3), Above Normal(4), High(5), Realtime(6) = ";
        cin >> mode;

        while (mode > 7 || mode < 0)
```

```
        while (mode > 7 || mode < 0)
        {
            cin >> mode;
        }

        switch (mode) {
        case 1:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_LOWEST);
            break;
        case 2:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_BELOW_NORMAL);
            break;
        case 3:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_NORMAL);
            break;
        case 4:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_ABOVE_NORMAL);
            break;
        case 5:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_HIGHEST);
            break;
        case 6:
            SetThreadPriority(hThreads[i], THREAD_PRIORITY_TIME_CRITICAL);
            break;
        }

        //Getting Thread Priority
        ThreadPriority = GetThreadPriority(thread);

        cout << "Thread Priority = " << ThreadPriority << endl;

```

```
        //Closing Handle of Thread
        CloseHandle(thread);

        cout << "Interlock increment = " << dwIncre << endl;

        last_interation += ThreadFuncTakeNums.size();
        cout << endl;
    }

    for (int i = 0; i < thread_num; ++i) {
        //WaitForSingleObject for Thread
        WaitForSingleObject(hThreadOpen[i], INFINITE);
        CloseHandle(hThreadOpen[i]);
    }

    vector<int> completed = complete(arr, elem_num, thread_num);
    cout << "Sorting number:" << endl;
    for (int i = 0; i < elem_num; ++i) {
        cout << completed[i] << " ";
    }
    cout << endl;

    system("pause");
    return 0;
}
```

Динамічний зв’язок між програмою та бібліотекою:

```
#include <iostream>
#include <windows.h>

using namespace std;

int main() {
    HMODULE lib;
    lib = LoadLibraryA("C:\\Users\\User\\source\\repos\\DLL1\\Debug\\DLL1.dll");

    if (lib != NULL)
    {
        void(*function)();
        (FARPROC&)function = GetProcAddress(lib, "selectionSort_start");
        function();
    }

    else
    {
        cout << "The error occured while loading dll file!" << endl;
        system("pause");
        return 0;
    }
    FreeLibrary(lib);
}
```

Статичний зв’язок між програмою та бібліотекою:

```
#include <iostream>
#include "Synchronization.h"

using namespace std;

int main() {
    selectionSort_start();

    system("pause");
    return 0;
}
```

Запустив створену бібліотеку з командної стрічки (cmd.exe) за допомогою rundll32.exe:

C:\Windows\system32\cmd.exe

```
Microsoft Windows [Version 10.0.17134.1304]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\User>cd C:\Users\User\source\repos\DLL1\Debug

C:\Users\User\source\repos\DLL1\Debug>rundll32 DLL1.dll, selectionSort_start

C:\Users\User\source\repos\DLL1\Debug>
```

C:\Windows\SysWOW64\rundll32.exe

```
Enter the number of elements: 10
6 5 1 2 3 1 3 1 3 3
Enter the number of threads: 2
Number of thread - 1
Thread ID = 12504
Values, which take this 12504 thread:
6 5 1 2 3
Set priority:
Enter index of priority Idle(1), Below Normal(2), Normal(3), Above Normal(4), High(5), Realtime(6) = 3
Thread Priority = 0
Interlock increment = 1

Number of thread - 2
Thread ID = 1708
Values, which take this 1708 thread:
1 3 1 3 3
Set priority:
Enter index of priority Idle(1), Below Normal(2), Normal(3), Above Normal(4), High(5), Realtime(6) = 3
Thread Priority = 0
Interlock increment = 2

Sorting number:
6 5 3 3 3 3 2 1 1 1
Press any key to continue . . .
```

Висновок: я ознайомився з динамічно-зв’язувальними бібліотеками (Dynamic-link library) в ОС Windows. Навчитися реалізовувати динамічно-зв’язувальні бібліотеки. Викликати їх через rundll32, встановлювати статичний зв’язок між програмою та бібліотекою та динамічний зв’язок між програмою та бібліотекою.